70610301 Kompyuter injiniringi
Sun'iy intellektga asoslangan kompyuter tizimlari muhandisligi

LABORATORIYA ISHI · MT2

AI modelini optimallashtirish: latency, throughput va energiya samaradorligi tahlili (ONNX Runtime, TensorRT, OpenVINO)

Bajarish bo‘yicha qo‘llanma: 7 bosqich · 10 mashq · 20 mustaqil variant

Kodlar va topshiriqlar: https://github.com/sayka7/ktm_labs/tree/main/MT2
Fan sayti (virtual laboratoriya): https://aiscan.airi.uz/ktm/#/c/mis2104/lab/MT2
O‘qituvchi: S.Iskandarova, t.f.f.d. (PhD), dotsent
2026-2027 o‘quv yili

Laboratoriya haqida

Bu mustaqil ishda akselerometr signallari bo'yicha harakat turini (yurish, yugurish, zina, yiqilish) aniqlovchi depthwise-separable 1D CNN chekka qurilma va server uchun optimallashtiriladi: to'g'ri o'lchov metodikasi, ONNX eksport va operatorlar profili, FP16/BF16/INT8 formatlari, dinamik paketlash va navbat simulyatsiyasi, parallellik (oqimlar, sessiyalar), energiya modeli va DVFS, yakuniy Pareto tahlili. Real labdan farqi: GPU, TensorRT va OpenVINO yo'q — ONNX Runtime CPU'da o'lchanadi, TensorRT/OpenVINO'ning mos sozlamalari va buyruqlari izoh sifatida beriladi, energiya esa quvvat modeli bilan baholanadi (raqamlar taxminiy).

> **Muhit.** Muhit: Python 3.12, CPU (2 yadro, 1,5 GB), tarmoq yo'q, vaqt chegarasi 90 s. Kutubxonalar: torch, torchvision, numpy, pandas, scipy, scikit-learn, matplotlib, onnx, onnxruntime, fairlearn, shap, numba, psutil, pyyaml, fastapi, httpx.

Bajarish tartibi

1. Kodlarni oling: `git clone https://github.com/sayka7/ktm_labs.git` (yoki GitHub’dan «Download ZIP», yoki saytdagi ZIP) va `cd ktm_labs/MT2`.
2. Kutubxonalar: `python3 -m venv venv && source venv/bin/activate`, so‘ng `pip install -r ../requirements.txt` (bir marta, ~2 GB).
3. Har bosqichni ketma-ket bajaring: `python3 s1_*.py`, natijani shu qo‘llanmadagi «Mening natijam» joyiga yozing yoki hisobotga qo‘shing.
4. Saytda ham bajarish mumkin (o‘rnatishsiz): https://aiscan.airi.uz/ktm/ → Mavzular → MT2 → 🧪 Virtual bajarish.
5. Mashqlar va o‘z variantingizni (jurnal raqami bo‘yicha) bajarib, hisobotni topshiring.

Mundarija

1. 1-qadam. Etalon model va o'lchov metodikasi
2. 2-qadam. ONNX eksport, grafik optimallashtirish va operatorlar profili
3. 3-qadam. Hisoblash aniqligi formatlari: FP32, FP16/BF16, INT8 dinamik va statik
4. 4-qadam. Paket va oqim tahlili: S(b) modeli va dinamik paketlash simulyatsiyasi
5. 5-qadam. Parallellik: intra-op oqimlar, sessiyalar va LATENCY/THROUGHPUT rejimlari
6. 6-qadam. Energiya samaradorligi: J/inferens, EDP, DVFS va quvvat rejimlari
7. 7-qadam. Yakuniy Pareto tahlili, ssenariy bo'yicha tanlov va TensorRT/OpenVINO yo'l xaritasi
8. Qo‘shimcha mashqlar (10)
9. Mustaqil topshiriqlar — 20 variant

1 1-qadam. Etalon model va o'lchov metodikasi

MAQSAD

Sintetik HAR ma'lumoti (3 o'q x 128 nuqta, 6 sinf, sensor shovqini bilan) yaratiladi va HARNet (depthwise-separable 1D CNN, ~8 ming parametr) 5 epoxa o'qitiladi. Asosiy e'tibor — **to'g'ri o'lchash**: taymer aniqligi, isinish (birinchi chaqiruv sekin), bir konfiguratsiyani 5 marta takrorlab p50/p95/p99/p99.9 va variatsiya koeffitsienti (CV). PyTorch eager va TorchScript (trace + freeze) solishtiriladi; kechikish gistogrammasi uzun dumni ko'rsatadi.

> **Kutilgan natija:** Test aniqligi ~0,92; 1-chaqiruv ~1,5 ms, keyingilari ~0,55 ms; TorchScript p50 ~0,37 ms (eager ~0,56 ms), CV < 1% (taxminiy).

DASTUR KODI (FAYL: S1_*.PY)

```
# 1-qadam. Etalon model (1D CNN, akselerometr) va to'g'ri o'lchov metodikasi: isinish,
takrorlar, foizlar, tarqalish
import warnings; warnings.filterwarnings("ignore")
import time, numpy as np, pandas as pd, torch, torch.nn as nn
from sklearn.model_selection import train_test_split
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

torch.manual_seed(0); np.random.seed(0)
torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
nn.MaxPool1d(2),
                               ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
```

```
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

X, y = har_yarat(3000)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
t0 = time.perf_counter(); model = oqit(HARNet(), Xtr, ytr); t_oq = time.perf_counter() - t0
with torch.no_grad():
    acc = (model(torch.tensor(Xte)).argmax(1).numpy() == yte).mean()
n_par = sum(p.numel() for p in model.parameters())
print("=== Etalon model (PyTorch FP32) ===")
print(f"HARNet: {n_par} parametr ({n_par * 4 / 1024:.1f} KB FP32), o'qitish {t_oq:.1f} s, test
aniqligi {acc:.3f}")
print("Sinflar:", ", ".join(SINFLAR))

# --- O'lchov metodikasi
print("\n=== O'lchov metodikasi ===")
print(f"Taymer aniqligi (perf_counter): {time.get_clock_info('perf_counter').resolution *
1e9:.0f} ns")
x1 = torch.tensor(Xte[:1])
def chaqir(fn, n):
    t = np.empty(n)
    for i in range(n):
        a = time.perf_counter(); fn(); t[i] = time.perf_counter() - a
    return t * 1e3
m_fresh = HARNet().eval()
with torch.inference_mode():
    sovuq = chaqir(lambda: m_fresh(x1), 50)
print(f"Isinish effekti: 1-chaqiruv {sovuq[0]:.2f} ms, 2–10 o'rtacha {sovuq[1:10].mean():.3f}
ms, 40–50 o'rtacha {sovuq[40:].mean():.3f} ms")

# Bir xil konfiguratsiyani 5 marta o'lchash — natijalar tarqalishi (o'rtacha emas, foizlar va
CV)
def toliq_olch(fn, takror=400, isinish=50):
    chaqir(fn, isinish); t = chaqir(fn, takror)
    return [np.percentile(t, q) for q in (50, 95, 99, 99.9)] + [t.mean()]
rows = []
with torch.inference_mode():
    jit = torch.jit.trace(model, x1); jit = torch.jit.freeze(jit)
    for nom, fn in [("eager + no_grad", lambda: model(x1)), ("TorchScript freeze", lambda:
jit(x1))]:
        for k in range(5):
            rows.append([nom, k + 1] + toliq_olch(fn))
df = pd.DataFrame(rows, columns=["Rejim", "Takror", "p50", "p95", "p99", "p99.9", "o'rtacha"])
print(df.round(3).to_string(index=False))
agg = df.groupby("Rejim")[["p50", "p99"]].agg(["mean", "std"])
for rj in agg.index:
    m50, s50 = agg.loc[rj, ("p50", "mean")], agg.loc[rj, ("p50", "std")]
    print(f"{rj}: p50 = {m50:.3f} ± {s50:.3f} ms (CV {s50 / m50:.1%}), p99 = {agg.loc[rj,
('p99', 'mean')]:.3f} ms")
print("Qoida: isinishsiz o'lchamang, kamida 3–5 marta takrorlang, o'rtacha emas p50/p99 ni va
tarqalishni (CV) bering.")

with torch.inference_mode():
    t = chaqir(lambda: model(x1), 2000)
```

```
plt.figure(figsize=(6, 3.2)); plt.hist(t, bins=80, color="tab:blue"); plt.yscale("log")
for q, c in [(50, "g"), (99, "r")]: plt.axvline(np.percentile(t, q), c=c, ls="--",
label=f"p{q}")
plt.xlabel("Kechikish, ms (batch 1)"); plt.ylabel("soni"); plt.title("Kechikish taqsimoti: uzun
dum"); plt.legend(); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Etalon model (PyTorch FP32) ===`, `=== O'lchov metodikasi ===`, `CV`

TAJRIBA QILIB KO‘RING

- Isinishni 0 ga tushirib, takror sonini 50 qiling — p99 qanday buziladi?
- torch.set_num_threads(1) bilan qayta o'lchang.
- HARNet(w=64) bilan parametrlar, aniqlik va kechikish qanday o'zgaradi?

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

2 2-qadam. ONNX eksport, grafik optimallashtirish va operatorlar profili

MAQSAD

Model ONNX'ga (opset 17, dinamik batch) eksport qilinadi. ORT ning 4 ta optimallashtirish darajasi uchun optimallashtirilgan grafik faylga saqlanib, tugunlar va operatorlar ro'yxati (Conv+Relu → FusedConv) solishtiriladi — bu TensorRT'ning engine qurishdagi va OpenVINO'ning kompilyatsiyadagi qatlam birlashtirishiga o'xshaydi. So'ngra `enable_profiling` bilan har operatorning vaqt ulushi hisoblanadi (trtexec --dumpProfile analogi) va Amdahl qonuni bo'yicha optimallashtirish nuqtasi aniqlanadi.

> **Kutilgan natija:** 14 tugun → 11 (FusedConv:3); ORT PyTorch eager'dan ~8 marta tez (~0,07 ms); profilda Conv/FusedConv vaqtning ~80% ini oladi (taxminiy).

DASTUR KODI (FAYL: S2_*.PY)

```
# 2-qadam. ONNX eksport, ORT grafik optimallashtirish (fusion) va operatorlar bo'yicha profil
import warnings; warnings.filterwarnings("ignore")
import os, json, time, glob, numpy as np, pandas as pd, torch, torch.nn as nn, onnx,
onnxruntime as ort

torch.manual_seed(0); np.random.seed(0)
torch.set_num_threads(2); torch.backends.nnpack.set_flags(False)
ort.set_default_logger_severity(3)                # ORT ogohlantirishlarini yashiramiz

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
nn.MaxPool1d(2),
```

```
                                ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

# Kechikish va profil vaznlarga bog'liq emas — bu qadamda model o'qitilmaydi (faqat
arxitektura)
model = HARNet().eval(); x1 = torch.randn(1, 3, 128)
torch.onnx.export(model, x1, "har.onnx", input_names=["x"], output_names=["logits"],
                  dynamic_axes={"x": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17,
dynamo=False)
m = onnx.load("har.onnx"); onnx.checker.check_model(m)
def operatorlar(mm):
    s = pd.Series([n.op_type for n in mm.graph.node]).value_counts(); return {k: int(v) for k,
v in s.items()}
print("=== ONNX eksport ===")
print(f"Fayl: {os.path.getsize('har.onnx') / 1024:.1f} KB, tugunlar: {len(m.graph.node)},
operatorlar: {operatorlar(m)}")

# --- Optimallashtirilgan grafikni faylga saqlab, fusion natijasini ko'ramiz (TensorRT «layer
fusion» ning CPU analogi)
rows = []
xb = np.random.randn(1, 3, 128).astype(np.float32)
for nom, lv in [("DISABLE_ALL", ort.GraphOptimizationLevel.ORT_DISABLE_ALL), ("BASIC",
ort.GraphOptimizationLevel.ORT_ENABLE_BASIC),
                ("EXTENDED", ort.GraphOptimizationLevel.ORT_ENABLE_EXTENDED), ("ALL",
ort.GraphOptimizationLevel.ORT_ENABLE_ALL)]:
    so = ort.SessionOptions(); so.graph_optimization_level = lv; so.intra_op_num_threads = 1
    so.optimized_model_filepath = f"har_{nom}.onnx"
    s = ort.InferenceSession("har.onnx", so, providers=["CPUExecutionProvider"])
    for _ in range(50): s.run(None, {"x": xb})
    t = []
    for _ in range(500):
        a = time.perf_counter(); s.run(None, {"x": xb}); t.append(time.perf_counter() - a)
    t = np.array(t) * 1e3; om = onnx.load(f"har_{nom}.onnx")
    rows.append((nom, len(om.graph.node), np.percentile(t, 50), np.percentile(t, 99), ",
".join(f"{k}:{v}" for k, v in operatorlar(om).items())))
print("\n=== Optimallashtirish darajalari (batch 1, 1 oqim) ===")
print(pd.DataFrame(rows, columns=["Daraja", "Tugun", "p50, ms", "p99, ms",
"Operatorlar"]).round(4).to_string(index=False))
with torch.inference_mode():
    tt = []
    for i in range(550):
        a = time.perf_counter(); model(x1); tt.append(time.perf_counter() - a)
pt50 = np.percentile(np.array(tt[50:]) * 1e3, 50)
print(f"Taqqos: PyTorch eager p50 = {pt50:.4f} ms -> ORT ALL {pt50 / rows[-1][2]:.1f} marta tez
(Python/dispatcher ustamasi yo'q)")
print("Izoh: BatchNorm eksportdayoq (eval + constant folding) Conv vazniga qo'shilgan; EXTENDED
da Conv+Relu -> FusedConv.")
print("TensorRT ham xuddi shunday (va kengroq) qatlam birlashtirishni engine qurishda, OpenVINO
esa model kompilyatsiyasida bajaradi.")

# --- Profil: qaysi operator qancha vaqt oladi (TensorRT: trtexec --dumpProfile, OpenVINO:
```

```
benchmark_app -pc analogi)
so = ort.SessionOptions(); so.enable_profiling = True; so.intra_op_num_threads = 1
s = ort.InferenceSession("har.onnx", so, providers=["CPUExecutionProvider"])
xb64 = np.random.randn(64, 3, 128).astype(np.float32)
for _ in range(100): s.run(None, {"x": xb64})
prof = s.end_profiling()
ev = [e for e in json.load(open(prof)) if e.get("cat") == "Node" and
e["name"].endswith("_kernel_time")]
df = pd.DataFrame([(e["args"]["op_name"], e["dur"]) for e in ev], columns=["op", "dur_us"])
agg = df.groupby("op").dur_us.agg(["sum", "count"]).sort_values("sum", ascending=False)
agg["ulush"] = agg["sum"] / agg["sum"].sum(); agg["o'rtacha_us"] = agg["sum"] / agg["count"]
print("\n=== ORT profil (operatorlar bo'yicha, batch 64, 100 chaqiruv) ===")
print(agg[["count", "o'rtacha_us", "ulush"]].round(3).to_string())
eng = agg.index[0]
print(f"Eng og'ir operator: {eng} ({agg.ulush.iloc[0]:.0%}) — optimallashtirishni shundan
boshlash kerak (Amdahl qonuni).")
for f in glob.glob("onnxruntime_profile_*.json"): os.remove(f)
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== ONNX eksport ===`, `=== Optimallashtirish darajalari`, `=== ORT profil`

TAJRIBA QILIB KO‘RING

- `do_constant_folding=False` bilan eksport qiling — BatchNorm tugunlari paydo bo'ladimi?
- Profilni batch 1 da qiling: operatorlar ulushi qanday o'zgaradi (ustama ulushi)?
- MaxPool o'rniga stride=2 li Conv ishlatib, profil va aniqlikni solishtiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

3 3-qadam. Hisoblash aniqligi formatlari: FP32, FP16/BF16, INT8 dinamik va statik

MAQSAD

O'qitilgan model 6 ko'rinishda solishtiriladi: ORT FP32, FP16 vazn yaxlitlash simulyatsiyasi (TensorRT --fp16 / OpenVINO FP16 IR ning sonli ta'siri), INT8 dinamik, INT8 statik QDQ (MinMax va Percentile) va QOperator (QLinearConv) formatlari, qo'shimcha ravishda PyTorch BF16 autocast. Aniqlikdan tashqari kvantlash xatosining sezgirroq o'lchovlari — FP32 bilan bashorat mosligi, max|Δlogit| va KL divergensiya — hamda hajm, batch 1 kechikishi va batch 64 throughput o'lchanadi.

> **Kutilgan natija:** Aniqlik barcha formatlarda ~0,92 (±0,5 p.p.), moslik >= 99%; INT8 statik hajmi ~16–24 KB (FP32 ~33 KB), throughput ~1,9x; dinamik INT8 bu modelda ~0,4x sekin (taxminiy).

DASTUR KODI (FAYL: S3_*.PY)

```
# 3-qadam. Hisoblash aniqligi formatlari: FP32, FP16/BF16 (simulyatsiya), INT8 dinamik va
statik (QDQ / QOperator)
import warnings; warnings.filterwarnings("ignore")
import os, io, time, logging, contextlib, numpy as np, pandas as pd, torch, torch.nn as nn,
onnx, onnxruntime as ort
from onnx import numpy_helper
from onnxruntime.quantization import (quantize_dynamic, quantize_static, QuantType,
QuantFormat, CalibrationDataReader, CalibrationMethod)
from onnxruntime.quantization.shape_inference import quant_pre_process
from sklearn.model_selection import train_test_split

logging.disable(logging.WARNING); ort.set_default_logger_severity(3)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
```

```
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
nn.MaxPool1d(2),
                               ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

X, y = har_yarat(3000)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
model = oqit(HARNet(), Xtr, ytr)
torch.onnx.export(model, torch.tensor(Xte[:1]), "fp32.onnx", input_names=["x"], output_names=
["logits"],
                  dynamic_axes={"x": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17,
dynamo=False)
quant_pre_process("fp32.onnx", "prep.onnx")

# --- FP16 vazn simulyatsiyasi: vaznlarni float16 ga yaxlitlab qaytaramiz (TensorRT --fp16 /
OpenVINO FP16 IR ning sonli ta'siri)
m = onnx.load("fp32.onnx")
for init in m.graph.initializer:
    w = numpy_helper.to_array(init)
    if w.dtype == np.float32:
init.CopyFrom(numpy_helper.from_array(w.astype(np.float16).astype(np.float32), init.name))
onnx.save(m, "fp16sim.onnx")

class Kalib(CalibrationDataReader):
    def __init__(self, data): self.it = iter([{"x": data[i:i + 32]} for i in range(0,
len(data), 32)])
    def get_next(self): return next(self.it, None)
kal = Xtr[:320]
quantize_dynamic("prep.onnx", "int8_dyn.onnx", weight_type=QuantType.QInt8)
quantize_static("prep.onnx", "int8_qdq_minmax.onnx", Kalib(kal), quant_format=QuantFormat.QDQ,
per_channel=True,
                activation_type=QuantType.QInt8, weight_type=QuantType.QInt8,
calibrate_method=CalibrationMethod.MinMax)
with contextlib.redirect_stdout(io.StringIO()):            # kalibrlash jarayoni chiqaradigan
matnni yashiramiz
    quantize_static("prep.onnx", "int8_qdq_pct.onnx", Kalib(kal), quant_format=QuantFormat.QDQ,
per_channel=True,
                    activation_type=QuantType.QInt8, weight_type=QuantType.QInt8,
calibrate_method=CalibrationMethod.Percentile,
                    extra_options={"CalibPercentile": 99.99})
quantize_static("prep.onnx", "int8_qop.onnx", Kalib(kal), quant_format=QuantFormat.QOperator,
per_channel=True,
                activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
calibrate_method=CalibrationMethod.MinMax)

def sess(f):
    so = ort.SessionOptions(); so.intra_op_num_threads = 1
    return ort.InferenceSession(f, so, providers=["CPUExecutionProvider"])
def lat(s, xb, n):
```

```
    for _ in range(20): s.run(None, {"x": xb})
    t = []
    for _ in range(n):
        a = time.perf_counter(); s.run(None, {"x": xb}); t.append(time.perf_counter() - a)
    return np.array(t)
def softmax(z): e = np.exp(z - z.max(1, keepdims=True)); return e / e.sum(1, keepdims=True)

ref = sess("fp32.onnx").run(None, {"x": Xte})[0]; pref = softmax(ref)
rows = []
cfg = [("ORT FP32", "fp32.onnx"), ("FP16 vazn (simul.)", "fp16sim.onnx"), ("INT8 dinamik",
"int8_dyn.onnx"),
       ("INT8 statik QDQ MinMax", "int8_qdq_minmax.onnx"), ("INT8 statik QDQ Percentile",
"int8_qdq_pct.onnx"), ("INT8 statik QOperator", "int8_qop.onnx")]
for nom, f in cfg:
    s = sess(f); out = s.run(None, {"x": Xte})[0]; p = softmax(out)
    kl = float(np.mean(np.sum(pref * np.log((pref + 1e-9) / (p + 1e-9)), 1)))
    t1 = lat(s, Xte[:1], 300); t64 = lat(s, Xte[:64], 40)
    rows.append((nom, os.path.getsize(f) / 1024, (out.argmax(1) == yte).mean(), (out.argmax(1)
== ref.argmax(1)).mean(),
                 np.abs(out - ref).max(), kl, np.percentile(t1, 50) * 1e3, 64 /
np.median(t64)))
# BF16: PyTorch autocast (Intel AMX / ARM bf16 li CPU'larda tezlashadi; bu yerda faqat sonli
ta'sir)
with torch.inference_mode(), torch.autocast("cpu", dtype=torch.bfloat16):
    ob = model(torch.tensor(Xte)).float().numpy()
pb = softmax(ob)
rows.append(("BF16 autocast (PyTorch)", np.nan, (ob.argmax(1) == yte).mean(), (ob.argmax(1) ==
ref.argmax(1)).mean(),
             np.abs(ob - ref).max(), float(np.mean(np.sum(pref * np.log((pref + 1e-9) / (pb +
1e-9)), 1))), np.nan, np.nan))
print("=== Aniqlik formatlari taqqosi ===")
df = pd.DataFrame(rows, columns=["Konfiguratsiya", "Hajm, KB", "Aniqlik", "FP32 bilan moslik",
"max|dlogit|", "KL", "p50 b1, ms", "inf/s b64"])
print(df.round(4).to_string(index=False))
print("\n=== Operatorlar (statik QDQ va QOperator) ===")
for f in ("int8_qdq_minmax.onnx", "int8_qop.onnx"):
    ops = pd.Series([n.op_type for n in onnx.load(f).graph.node]).value_counts()
    print(f, {k: int(v) for k, v in ops.items()})
print("Izoh: «moslik» va KL — kvantlash xatosining aniqlikdan sezgirroq o'lchovi (aniqlik
tasodifan saqlanib qolishi mumkin).")
tz = df.set_index("Konfiguratsiya")["inf/s b64"]
print(f"Bu CPU'da: statik INT8 throughput FP32 ga nisbatan {tz['INT8 statik QOperator'] /
tz['ORT FP32']:.1f}x, dinamik INT8 {tz['INT8 dinamik'] / tz['ORT FP32']:.1f}x")
print("(dinamik kvantlash har chaqiruvda faollashuv masshtabini hisoblaydi — kichik Conv
modelda bu ustama foydadan katta).")
print("VNNI/AMX li Intel CPU (OpenVINO) yoki Tensor Core'li GPU (TensorRT) da INT8 farqi odatda
kattaroq bo'ladi (taxminiy, o'lchab ko'rish shart).")
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Aniqlik formatlari taqqosi ===`, `QLinearConv`, `INT8 statik QDQ MinMax`

TAJRIBA QILIB KO‘RING

- Kalibrlash to'plamini 320 dan 32 ga tushiring — moslik va KL qanday o'zgaradi?
- `per_channel=False` bilan statik kvantlang va depthwise Conv xatosini solishtiring.
- `CalibrationMethod.Entropy` qo'shing va uch kalibrlash usulini taqqoslang.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

4 4-qadam. Paket va oqim tahlili: S(b) modeli va dinamik paketlash simulyatsiyasi

MAQSAD

Batch 1–128 uchun xizmat vaqti o'lchanadi va chiziqli model S(b) = c0 + c1·b moslanadi (c0 — har chaqiruv ustamasi, c1 — namuna narxi). So'ngra Puasson oqimi bilan navbat simulyatsiyasi: paketsiz, dinamik paketlash (b<=16, kutish 0,3 ms) va (b<=64, kutish 1 ms) siyosatlari turli yuklamada p50/p99 va throughput bo'yicha solishtiriladi (Triton dynamic_batching, OpenVINO avtomatik paketlash g'oyasi). Xuddi shu simulyatsiya GPU uslubidagi S(b) (katta c0, kichik c1 — faraziy) uchun takrorlanadi.

> **Kutilgan natija:** Bu CPU'da c0 kichik (~0,02 ms), throughput batch bilan ~15k → ~18,5k inf/s; GPU uslubida dinamik paketlash SLO ichidagi yuklamani ~1000 dan ~60 000 so'rov/s gacha oshiradi (taxminiy).

DASTUR KODI (FAYL: S4_*.PY)

```
# 4-qadam. Paket (batch) va oqim tahlili: xizmat vaqti modeli S(b), dinamik paketlash
simulyatsiyasi, SLO bo'yicha maksimal yuklama
import warnings; warnings.filterwarnings("ignore")
import time, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

ort.set_default_logger_severity(3)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
```

```
nn.MaxPool1d(2),
                                ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

model = HARNet().eval()
torch.onnx.export(model, torch.randn(1, 3, 128), "har.onnx", input_names=["x"], output_names=
["logits"],
                  dynamic_axes={"x": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17,
dynamo=False)
so = ort.SessionOptions(); so.intra_op_num_threads = 1
s = ort.InferenceSession("har.onnx", so, providers=["CPUExecutionProvider"])

# --- 1) Xizmat vaqti S(b): paket o'lchami bo'yicha o'lchov
rows = []
for b in (1, 2, 4, 8, 16, 32, 64, 128):
    xb = np.random.randn(b, 3, 128).astype(np.float32)
    for _ in range(10): s.run(None, {"x": xb})
    t = []
    for _ in range(max(30, 600 // b)):
        a = time.perf_counter(); s.run(None, {"x": xb}); t.append(time.perf_counter() - a)
    t = np.array(t) * 1e3
    rows.append((b, np.median(t), np.percentile(t, 99), b / np.median(t) * 1e3))
S = pd.DataFrame(rows, columns=["batch", "S_p50, ms", "S_p99, ms", "inf/s"])
c1, c0 = np.polyfit(S.batch, S["S_p50, ms"], 1)                    # S(b) ~ c0 + c1*b
print("=== Paket o'lchami: xizmat vaqti va throughput ===")
print(S.round(4).to_string(index=False))
print(f"Chiziqli model: S(b) = {c0:.4f} + {c1:.4f}*b ms  (c0 — har chaqiruvning qat'iy
ustamasi, c1 — namuna narxi)")
print(f"Nazariy throughput chegarasi: 1/c1 = {1 / c1 * 1e3:.0f} inf/s; batch 1 da: {1 / (c0 +
c1) * 1e3:.0f} inf/s")

# --- 2) Dinamik paketlash simulyatsiyasi (Triton/TensorRT-LLM/OpenVINO «auto-batching»
g'oyasi)
def simul(lam, max_b, max_wait_ms, c0, c1, n=6000, seed=0):
    """Puasson oqimi (lam so'rov/s), bitta ishchi. Navbatdan max_b gacha olinadi; birinchi
so'rov max_wait kutsa — jo'natiladi."""
    r = np.random.default_rng(seed)
    kel = np.cumsum(r.exponential(1000.0 / lam, n))                  # kelish vaqtlari, ms
    bosh, i, kech, bs = 0.0, 0, np.empty(n), []
    while i < n:
        start = max(bosh, kel[i])                                    # ishchi bo'shagan va
so'rov kelgan payt
        t_lim = max(start, kel[i] + max_wait_ms)                     # paketni to'ldirish
uchun kutish chegarasi
        j = min(i + max_b, int(np.searchsorted(kel, t_lim, side="right")))
        launch = max(start, kel[j - 1]) if j - i == max_b else t_lim  # paket to'lsa — darhol,
aks holda muddat tugaganda
        tugash = launch + c0 + c1 * (j - i)
        kech[i:j] = tugash - kel[i:j]; bs.append(j - i); bosh = tugash; i = j
    return np.percentile(kech, 50), np.percentile(kech, 99), np.mean(bs), n / (bosh - kel[0]) *
1e3
```

```
SLO = 2.0                                                    # p99 <= 2 ms (taxminiy
talab)
SIYOSAT = [("paketsiz", 1, 0.0), ("dinamik b<=16, kutish 0,3 ms", 16, 0.3), ("dinamik b<=64,
kutish 1 ms", 64, 1.0)]
def jadval(c0_, c1_, yuklamalar):
    rows = []
    for lam in yuklamalar:
        for nom, mb, mw in SIYOSAT:
            p50, p99, ob, X = simul(lam, mb, mw, c0_, c1_)
            rows.append((lam, nom, p50, p99, ob, X, "ha" if p99 <= SLO else "yo'q"))
    D = pd.DataFrame(rows, columns=["yuklama", "siyosat", "p50, ms", "p99, ms", "o'rt. paket",
"throughput", "SLO"])
    print(D.round(3).to_string(index=False))
    for nom, _, _ in SIYOSAT:
        ok = D[(D.siyosat == nom) & (D.SLO == "ha")]
        print(f"  {nom:30s}: SLO ni saqlagan maksimal yuklama ~ {ok.yuklama.max() if len(ok)
else 0} so'rov/s")
    return D
print(f"\n=== Dinamik paketlash simulyatsiyasi: bu CPU (o'lchangan S(b)), SLO p99 <= {SLO} ms
===")
D = jadval(c0, c1, (2000, 6000, 10000, 14000, 20000))
# GPU'da qat'iy ustama (kernel ishga tushirish, PCIe nusxalash) katta, namuna narxi kichik —
paketlash foydasi ancha katta
g0, g1 = 0.40, 0.004
print(f"\n=== Xuddi shu simulyatsiya: GPU uslubidagi S(b) = {g0} + {g1}*b ms (TAXMINIY,
illyustrativ) ===")
G = jadval(g0, g1, (1000, 2000, 5000, 20000, 60000))
print("Xulosa: S(b) dagi c0/c1 nisbati katta bo'lsa (GPU, TensorRT) dinamik paketlash
throughput'ni bir necha barobar oshiradi;")
print("bu CPU'da c0 kichik — paketlash faqat navbat to'lib qolganda yordam beradi. max_wait —
kechikish va throughput o'rtasidagi tutqich.")

fig, ax = plt.subplots(1, 2, figsize=(10, 3.3))
ax[0].plot(S.batch, S["inf/s"], marker="o"); ax[0].set_xscale("log", base=2);
ax[0].set_xlabel("batch"); ax[0].set_ylabel("inf/s"); ax[0].set_title("Throughput vs paket")
for nom in G.siyosat.unique():
    d = G[G.siyosat == nom]; ax[1].plot(d["yuklama"], d["p99, ms"].clip(upper=50), marker="o",
label=nom)
ax[1].axhline(SLO, ls="--", c="r"); ax[1].set_yscale("log"); ax[1].set_xlabel("yuklama,
so'rov/s"); ax[1].set_ylabel("p99, ms (GPU uslubi)"); ax[1].set_xscale("log");
ax[1].legend(fontsize=7); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Paket o'lchami: xizmat vaqti va throughput ===`, `=== Dinamik paketlash simulyatsiyasi`, `GPU uslubidagi`

TAJRIBA QILIB KO‘RING

- max_wait ni 0,1 / 0,5 / 2 ms qilib, p99–throughput egri chizig'ini chizing.
- Puasson o'rniga «portlovchi» (bursty) oqim (masalan, har 10 ms da 50 so'rov) bilan simulyatsiya qiling.
- Ikki ishchi (2 nusxa) uchun simulyatorni kengaytiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

5 5-qadam. Parallellik: intra-op oqimlar, sessiyalar va LATENCY/THROUGHPUT rejimlari

MAQSAD

Kengroq model (w=64) uchun 5 konfiguratsiya batch 1 va 32 da o'lchanadi: 1 sessiya/1 oqim, intra=2, umumiy sessiyaga 2 Python oqimi, 2 sessiya x 1 oqim va ortiqcha obuna (intra=2 x 2 oqim). Natija OpenVINO'ning NUM_STREAMS va PERFORMANCE_HINT (LATENCY/THROUGHPUT), TensorRT'ning bir nechta execution context + CUDA stream tushunchalari bilan jadvalda bog'lanadi.

> **Kutilgan natija:** Eng past kechikish — intra=2 (1 oqim); eng yuqori throughput — 2 sessiya x 1 oqim (~2x); ortiqcha obunada p99 keskin oshadi (taxminiy).

DASTUR KODI (FAYL: S5_*.PY)

```
# 5-qadam. Parallellik: intra-op oqimlar, bir nechta so'rov oqimi (streams) va «latency» /
«throughput» rejimlari
import warnings; warnings.filterwarnings("ignore")
import time, threading, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
from concurrent.futures import ThreadPoolExecutor
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

ort.set_default_logger_severity(3)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
nn.MaxPool1d(2),
                               ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
```

```
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

# Kengroq model (w=64): oqimlararo parallellik sezilishi uchun hisob ko'proq bo'lsin
torch.onnx.export(HARNet(w=64).eval(), torch.randn(1, 3, 128), "har64.onnx", input_names=["x"],
output_names=["logits"],
                  dynamic_axes={"x": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17,
dynamo=False)
def sessiya(intra, inter=1, parallel=False):
    so = ort.SessionOptions(); so.intra_op_num_threads = intra; so.inter_op_num_threads = inter
    if parallel: so.execution_mode = ort.ExecutionMode.ORT_PARALLEL
    return ort.InferenceSession("har64.onnx", so, providers=["CPUExecutionProvider"])

def yuklama(sessiyalar, ishchi, batch, jami=1200):
    """ishchi ta Python oqimi navbatdagi so'rovlarni bajaradi; har ishchi o'z sessiyasini (yoki
umumiy sessiyani) ishlatadi."""
    xb = np.random.randn(batch, 3, 128).astype(np.float32)
    for s in sessiyalar:
        for _ in range(20): s.run(None, {"x": xb})
    lat, lock, hisob = [], threading.Lock(), [0]
    def ish(k):
        s = sessiyalar[k % len(sessiyalar)]; mahalliy = []
        while True:
            with lock:
                if hisob[0] >= jami // batch: break
                hisob[0] += 1
            a = time.perf_counter(); s.run(None, {"x": xb});
mahalliy.append(time.perf_counter() - a)
        with lock: lat.extend(mahalliy)
    t0 = time.perf_counter()
    with ThreadPoolExecutor(ishchi) as ex: list(ex.map(ish, range(ishchi)))
    dt = time.perf_counter() - t0; lat = np.array(lat) * 1e3
    return np.percentile(lat, 50), np.percentile(lat, 99), len(lat) * batch / dt

CFG = [("A: 1 sessiya, intra=1, 1 oqim (LATENCY)", lambda: [sessiya(1)], 1),
       ("B: 1 sessiya, intra=2, 1 oqim", lambda: [sessiya(2)], 1),
       ("C: 1 umumiy sessiya, intra=1, 2 oqim", lambda: [sessiya(1)], 2),
       ("D: 2 sessiya x intra=1, 2 oqim (THROUGHPUT)", lambda: [sessiya(1), sessiya(1)], 2),
       ("E: 1 sessiya, intra=2, 2 oqim (ortiqcha obuna)", lambda: [sessiya(2)], 2)]
rows = []
for batch in (1, 32):
    for nom, f, ish in CFG:
        p50, p99, X = yuklama(f(), ish, batch, jami=1200 if batch == 1 else 6400)
        rows.append((batch, nom, p50, p99, X))
df = pd.DataFrame(rows, columns=["batch", "Konfiguratsiya", "p50, ms", "p99, ms", "inf/s"])
df["nisbiy_X"] = df["inf/s"] / df.groupby("batch")["inf/s"].transform("first")
print("=== Parallellik konfiguratsiyalari (2 yadroli CPU) ===")
print(df.round(3).to_string(index=False))
for b in (1, 32):
    d = df[df.batch == b]; eng = d.loc[d["inf/s"].idxmax()]; tez = d.loc[d["p50, ms"].idxmin()]
    print(f"batch {b}: eng yuqori throughput — {eng.Konfiguratsiya} ({eng['inf/s']:.0f} inf/s);
eng past p50 — {tez.Konfiguratsiya}")
```

```
print("\n=== OpenVINO / TensorRT dagi analoglari ===")
print(pd.DataFrame([
    ("intra_op_num_threads", "INFERENCE_NUM_THREADS", "— (GPU ichida avtomatik)"),
    ("bir nechta sessiya / oqim", "NUM_STREAMS, PERFORMANCE_HINT=THROUGHPUT", "bir nechta
execution context + CUDA stream"),
    ("1 sessiya, 1 oqim", "PERFORMANCE_HINT=LATENCY", "1 context, batch 1"),
    ("dinamik paket (4-qadam)", "avtomatik paketlash (BATCH:GPU)", "Triton dynamic_batching"),
], columns=["ONNX Runtime (bu lab)", "OpenVINO", "TensorRT / Triton"]).to_string(index=False))
print("Qoida: yadrolar sonidan ko'p oqim (intra x ishchi > yadro) — ortiqcha obuna: p99 o'sadi,
throughput oshmaydi.")

d = df[df.batch == 32]
plt.figure(figsize=(7, 3.3)); plt.barh([k.split(":")[0] for k in d.Konfiguratsiya], d["inf/s"],
color="tab:green")
plt.xlabel("inf/s (batch 32)"); plt.title("Parallellik konfiguratsiyalari"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Parallellik konfiguratsiyalari`, `=== OpenVINO / TensorRT dagi analoglari ===`

TAJRIBA QILIB KO‘RING

- ORT_PARALLEL rejimi va inter_op_num_threads=2 ni qo'shib ko'ring — bu ketma-ket grafikda foyda beradimi?
- 3 va 4 ishchi oqim bilan sinab, throughput to'yinish nuqtasini toping.
- Batch 8 uchun ham jadvalni to'ldiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

6 6-qadam. Energiya samaradorligi: J/inferens, EDP, DVFS va quvvat rejimlari

MAQSAD

RAPL/nvidia-smi yo'qligi sababli quvvat modeli ishlatiladi: P = P_bosh + (P_max − P_bosh)·bandlik, bandlik = CPU vaqti / (devor vaqti·yadro). FP32 va INT8 statik, 1/2 oqim, batch 1/32 uchun µJ/inferens (to'liq va dinamik), inf/J va EDP (energiya x kechikish) hisoblanadi. So'ngra DVFS simulyatsiyasi: P_din ~ f·V², kechikish ~ 1/f — SLO ichida eng tejamkor chastota va «race-to-idle» sharti; oxirida Jetson uslubidagi nvpmodel rejimlari (faraziy raqamlar) bo'yicha tanlov.

> **Kutilgan natija:** Eng kam energiya — INT8 statik, 1 oqim, batch 32 (~500 µJ/inf, taxminiy); 2 oqim batch 1 da energiya oshadi; DVFS optimumi ~1,2 GHz, statik quvvat katta bo'lsa optimum yuqori chastotaga siljiydi.

DASTUR KODI (FAYL: S6_*.PY)

```
# 6-qadam. Energiya samaradorligi: CPU-vaqt asosidagi quvvat modeli, J/inferens, EDP, DVFS va
«race-to-idle» simulyatsiyasi
import warnings; warnings.filterwarnings("ignore")
import io, time, logging, contextlib, numpy as np, pandas as pd, torch, torch.nn as nn,
onnxruntime as ort
from onnxruntime.quantization import quantize_static, QuantType, QuantFormat,
CalibrationDataReader, CalibrationMethod
from onnxruntime.quantization.shape_inference import quant_pre_process
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

logging.disable(logging.WARNING); ort.set_default_logger_severity(3)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
```

```
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
nn.MaxPool1d(2),
                               ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

# Energiya vaznlarga bog'liq emas — o'qitmaymiz; INT8 uchun kalibrlash sintetik HAR
namunalarida
X, _ = har_yarat(400)
torch.onnx.export(HARNet().eval(), torch.tensor(X[:1]), "fp32.onnx", input_names=["x"],
output_names=["logits"],
                  dynamic_axes={"x": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17,
dynamo=False)
quant_pre_process("fp32.onnx", "prep.onnx")
class Kalib(CalibrationDataReader):
    def __init__(self): self.it = iter([{"x": X[i:i + 32]} for i in range(0, 320, 32)])
    def get_next(self): return next(self.it, None)
quantize_static("prep.onnx", "int8.onnx", Kalib(), quant_format=QuantFormat.QOperator,
per_channel=True,
                activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
calibrate_method=CalibrationMethod.MinMax)

# --- Quvvat modeli (RAPL/INA219 yo'q — TAXMINIY): P = P_bosh + (P_max - P_bosh) * bandlik,
bandlik = CPU vaqti / (devor vaqti * yadro)
P_BOSH, P_MAX, YADRO = 6.0, 28.0, 2           # W — noutbuk/kichik server CPU paketi uchun
faraz
def olch(fayl, intra, batch, n_inf=12800):
    so = ort.SessionOptions(); so.intra_op_num_threads = intra
    s = ort.InferenceSession(fayl, so, providers=["CPUExecutionProvider"])
    xb = np.random.randn(batch, 3, 128).astype(np.float32)
    for _ in range(20): s.run(None, {"x": xb})
    c0, w0 = time.process_time(), time.perf_counter()
    for _ in range(n_inf // batch): s.run(None, {"x": xb})
    cpu, wall = time.process_time() - c0, time.perf_counter() - w0
    band = min(cpu / (wall * YADRO), 1.0); P = P_BOSH + (P_MAX - P_BOSH) * band
    E = P * wall / n_inf                                          # J/inferens (bosh holat
quvvati bilan)
    Ed = (P_MAX - P_BOSH) * cpu / YADRO / n_inf                   # faqat dinamik
(marginal) qism
    lat = wall / (n_inf // batch) * 1e3
    return dict(bandlik=band, quvvat_W=P, lat_ms=lat, inf_s=n_inf / wall, uJ_inf=E * 1e6,
uJ_dinamik=Ed * 1e6,
                inf_J=1 / E, EDP=E * 1e6 * lat)
rows = []
for nom, f in (("FP32", "fp32.onnx"), ("INT8 statik", "int8.onnx")):
    for intra in (1, 2):
        for batch in (1, 32):
            rows.append(dict(model=nom, intra=intra, batch=batch, **olch(f, intra, batch)))
E = pd.DataFrame(rows)
```

```
print("=== Energiya bahosi (quvvat modeli TAXMINIY: P_bosh 6 W, P_max 28 W) ===")
print(E.round(3).to_string(index=False))
eng = E.loc[E.uJ_inf.idxmin()]; edp = E.loc[E.EDP.idxmin()]
print(f"Eng kam energiya: {eng.model}, intra={eng.intra}, batch={eng.batch} -> {eng.uJ_inf:.1f}
uJ/inf ({eng.inf_J:.0f} inf/J)")
print(f"Eng yaxshi EDP (energiya x kechikish): {edp.model}, intra={edp.intra}, batch=
{edp.batch}")
print("Izoh: 2 oqimda bandlik ~2x, tezlik esa < 2x bo'lsa — energiya/inferens oshadi; bosh
holat quvvati tez ishlashni rag'batlantiradi.")

# --- DVFS: chastota f pasayganda P_dinamik ~ f*V^2, kechikish ~ 1/f (hisobga bog'liq yuk uchun
faraz)
F = np.array([0.8, 1.2, 1.6, 2.0, 2.4, 2.8, 3.2])                  # GHz
V = 0.70 + (1.10 - 0.70) * (F - F[0]) / (F[-1] - F[0])           # kuchlanish, V (chiziqli
faraz)
P_STAT, P_DYN_MAX = 3.0, 22.0                                       # W: statik (oqish) va maks.
dinamik
Pd = P_DYN_MAX * (F / F[-1]) * (V / V[-1]) ** 2
base = E[(E.model == "FP32") & (E.intra == 1) & (E.batch == 1)].iloc[0]
lat_f = base.lat_ms * (F[-1] / F)                                   # o'lchangan kechikish maks.
chastotada deb faraz
yuk = 2000                                                           # so'rov/s — davriy yuk
band_f = np.minimum(yuk * lat_f / 1000, 1.0)                       # bir yadro bandligi
P_ish = P_STAT + Pd                                                 # ishlayotgandagi quvvat
P_bosh_holat = P_STAT + 0.05 * P_DYN_MAX                           # chuqur uyqu bo'lmagan bosh
holat
E_sek = band_f * P_ish + (1 - band_f) * P_bosh_holat                # 1 soniyalik energiya, J
D = pd.DataFrame(dict(f_GHz=F, V=V, P_ish_W=P_ish, lat_ms=lat_f, bandlik=band_f,
uJ_inf_faqat_ish=P_ish * lat_f * 1e3,
                      J_sekund=E_sek, uJ_inf_jami=E_sek / yuk * 1e6))
SLO = 0.25
D["SLO"] = np.where(D.lat_ms <= SLO, "ha", "yo'q")
print(f"\n=== DVFS simulyatsiyasi (yuk {yuk} so'rov/s, SLO kechikish <= {SLO} ms; parametrlar
FARAZIY) ===")
print(D.round(3).to_string(index=False))
ok = D[D.SLO == "ha"]; best = ok.loc[ok.uJ_inf_jami.idxmin()]
print(f"SLO ichida eng tejamkor chastota: {best.f_GHz} GHz ({best.uJ_inf_jami:.1f} uJ/inf);
maks. chastotada: {D.uJ_inf_jami.iloc[-1]:.1f} uJ/inf")
# Chuqur uyqu (C-state) mavjud bo'lsa bosh holat ~0,5 W; statik (oqish) quvvati faqat ish
paytida — u katta bo'lsa tez ishlash foydali
for p_stat in (1.0, 3.0, 10.0, 20.0):
    Eb = band_f * (p_stat + Pd) + (1 - band_f) * 0.5
    print(f"  statik quvvat {p_stat:4.1f} W, uyqu 0,5 W -> eng tejamkor chastota
{F[np.argmin(np.where(lat_f <= SLO, Eb, np.inf))]} GHz")
print("Race-to-idle: ish paytidagi statik quvvat katta va uyqu chuqur bo'lsa — tez ishlab
«uxlash», aks holda past chastotada sekin ishlash foydali.")

# --- Jetson uslubidagi quvvat rejimlari (nvpmodel) — FARAZIY illyustratsiya, real raqamlar
tegrastats bilan o'lchanadi
J = pd.DataFrame(dict(rejim=["10 W", "15 W", "25 W", "MAXN"], quvvat_W=[10.0, 15.0, 25.0,
40.0], nisbiy_tezlik=[0.45, 0.65, 0.85, 1.00]))
J["inf_s"] = 30000 * J.nisbiy_tezlik; J["inf_J"] = J.inf_s / J.quvvat_W
print("\n=== Jetson uslubidagi quvvat rejimlari (FARAZIY raqamlar) ===")
print(J.round(2).to_string(index=False))
kerak = 15000; mos = J[J.inf_s >= kerak]
print(f"Talab {kerak} inf/s: mos rejimlar {list(mos.rejim)}, eng tejamkori —
{mos.loc[mos.inf_J.idxmax()].rejim}")
print("Real stendda: sudo nvpmodel -m <rejim>; tegrastats yoki tashqi quvvat o'lchagich bilan W
ni yozib, inf/J hisoblanadi.")

fig, ax = plt.subplots(1, 2, figsize=(10, 3.3))
lab = E.model + " i" + E.intra.astype(str) + " b" + E.batch.astype(str)
ax[0].barh(lab, E.uJ_inf, color="tab:orange"); ax[0].set_xlabel("uJ/inferens (taxminiy)");
```

```
ax[0].set_title("Energiya/inferens")
ax[1].plot(D.f_GHz, D.uJ_inf_jami, marker="o"); ax[1].set_xlabel("Chastota, GHz");
ax[1].set_ylabel("uJ/inf (jami)"); ax[1].set_title("DVFS: optimal chastota");
plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Energiya bahosi`, `=== DVFS simulyatsiyasi`, `=== Jetson uslubidagi quvvat rejimlari`

TAJRIBA QILIB KO‘RING

- P_BOSH va P_MAX ni o'z noutbukingiz TDP'siga moslang — eng tejamkor konfiguratsiya o'zgaradimi?
- psutil.Process().cpu_times() bilan user/system vaqtni ajrating.
- DVFS da yuk 500 va 8000 so'rov/s bo'lganda optimal chastotani toping.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

7 7-qadam. Yakuniy Pareto tahlili, ssenariy bo'yicha tanlov va TensorRT/OpenVINO yo'l xaritasi

MAQSAD

6 konfiguratsiya (PyTorch eager, ORT FP32 1/2 oqim, INT8 dinamik, INT8 statik 1/2 oqim) bitta jadvalga yig'iladi: aniqlik, p99 (batch 1), throughput (batch 32), µJ/inferens va hajm. To'rt mezonli Pareto chegarasi aniqlanadi va uch ssenariy (bilakuzuk, kasalxona serveri, real vaqt signali) uchun cheklovlar va maqsad funksiyasi bo'yicha konfiguratsiya tanlanadi. Oxirida GPU/NPU li stendda bajariladigan trtexec, ovc/benchmark_app va quvvat o'lchash buyruqlari beriladi.

> **Kutilgan natija:** Pareto chegarasida INT8 statik konfiguratsiyalar; bilakuzuk uchun INT8 statik 1 oqim, server uchun eng yuqori throughput'li variant tanlanadi; grafikda qizil nuqtalar — Pareto (raqamlar taxminiy).

DASTUR KODI (FAYL: S7_*.PY)

```
# 7-qadam. Yakuniy tahlil: aniqlik-kechikish-energiya Pareto chegarasi, ssenariy bo'yicha
tanlov va TensorRT/OpenVINO yo'l xaritasi
import warnings; warnings.filterwarnings("ignore")
import os, time, logging, numpy as np, pandas as pd, torch, torch.nn as nn, onnxruntime as ort
from onnxruntime.quantization import quantize_dynamic, quantize_static, QuantType, QuantFormat,
CalibrationDataReader, CalibrationMethod
from onnxruntime.quantization.shape_inference import quant_pre_process
from sklearn.model_selection import train_test_split
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt

logging.disable(logging.WARNING); ort.set_default_logger_severity(3)
torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(2);
torch.backends.nnpack.set_flags(False)

# --- Sintetik akselerometr (HAR) ma'lumoti: 3 o'q x 128 nuqta (50 Hz, 2,56 s), 6 harakat sinfi
SINFLAR = ["tinch", "yurish", "yugurish", "zina_yuqori", "zina_past", "yiqilish"]
def har_yarat(n=3000, seed=0):
    r = np.random.default_rng(seed); t = np.arange(128) / 50.0
    X = np.zeros((n, 3, 128), np.float32); y = r.integers(0, 6, n)
    for j in range(n):
        c, f, ph = y[j], r.uniform(1.6, 2.4), r.uniform(0, 2 * np.pi)
        amp = {0: 0.05, 1: 0.6, 2: 1.0, 3: 0.65, 4: 0.65, 5: 0.3}[c] * r.uniform(0.7, 1.3)
        if c == 2: f *= 1.3
        s = amp * np.sin(2 * np.pi * f * t + ph)
        X[j, 0] = s; X[j, 1] = 0.5 * amp * np.sin(2 * np.pi * f * t + ph + 1.2); X[j, 2] = 1.0
+ 0.6 * amp * np.cos(2 * np.pi * 2 * f * t + ph)
        if c == 3: X[j, 2] += np.linspace(0, 0.4, 128)          # yuqoriga — vertikal tezlanish
trendi
        if c == 4: X[j, 2] -= np.linspace(0, 0.4, 128)
        if c == 5: k = r.integers(20, 100); X[j, :, k:k + 6] += r.normal(3.0, 0.8, (3, 1));
X[j, 2, k + 6:] -= 1.0   # zarba, keyin yotish
        X[j] += r.normal(0, 0.6, (3, 128))                      # sensor shovqini
    return X, y

# --- Model: depthwise-separable 1D CNN (MobileNet g'oyasi, chekka qurilmalar uchun)
class HARNet(nn.Module):
    def __init__(self, w=32):
        super().__init__()
```

```
        def ds(ci, co):  # depthwise (har kanal alohida) + pointwise (1x1) konvolyutsiya
            return nn.Sequential(nn.Conv1d(ci, ci, 5, padding=2, groups=ci), nn.Conv1d(ci, co,
1), nn.BatchNorm1d(co), nn.ReLU(), nn.MaxPool1d(2))
        self.f = nn.Sequential(nn.Conv1d(3, w, 5, padding=2), nn.BatchNorm1d(w), nn.ReLU(),
nn.MaxPool1d(2),
                               ds(w, 2 * w), ds(2 * w, 2 * w), nn.AdaptiveAvgPool1d(1),
nn.Flatten())
        self.head = nn.Linear(2 * w, 6)
    def forward(self, x):
        return self.head(self.f(x))

def oqit(model, Xtr, ytr, epoxa=5, lr=3e-3):
    opt = torch.optim.Adam(model.parameters(), lr); lf = nn.CrossEntropyLoss()
    Xt, yt = torch.tensor(Xtr), torch.tensor(ytr)
    for _ in range(epoxa):
        model.train(); perm = torch.randperm(len(Xt))
        for i in range(0, len(Xt), 64):
            b = perm[i:i + 64]; opt.zero_grad(); lf(model(Xt[b]), yt[b]).backward(); opt.step()
    return model.eval()

X, y = har_yarat(3000)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
model = oqit(HARNet(), Xtr, ytr)
torch.onnx.export(model, torch.tensor(Xte[:1]), "fp32.onnx", input_names=["x"], output_names=
["logits"],
                  dynamic_axes={"x": {0: "batch"}, "logits": {0: "batch"}}, opset_version=17,
dynamo=False)
quant_pre_process("fp32.onnx", "prep.onnx")
class Kalib(CalibrationDataReader):
    def __init__(self): self.it = iter([{"x": Xtr[i:i + 32]} for i in range(0, 320, 32)])
    def get_next(self): return next(self.it, None)
quantize_dynamic("prep.onnx", "dyn.onnx", weight_type=QuantType.QInt8)
quantize_static("prep.onnx", "int8.onnx", Kalib(), quant_format=QuantFormat.QOperator,
per_channel=True,
                activation_type=QuantType.QUInt8, weight_type=QuantType.QInt8,
calibrate_method=CalibrationMethod.MinMax)

P_BOSH, P_MAX, YADRO = 6.0, 28.0, 2                                  # 6-qadamdagi TAXMINIY
quvvat modeli
def olchov(run, batch):
    xb1, xbb = Xte[:1], Xte[:batch]
    for _ in range(30): run(xb1)
    t = []
    for _ in range(400):
        a = time.perf_counter(); run(xb1); t.append(time.perf_counter() - a)
    c0, w0 = time.process_time(), time.perf_counter(); k = 200
    for _ in range(k): run(xbb)
    cpu, wall = time.process_time() - c0, time.perf_counter() - w0
    band = min(cpu / (wall * YADRO), 1.0)
    return np.percentile(np.array(t) * 1e3, 99), k * batch / wall, (P_BOSH + (P_MAX - P_BOSH) *
band) * wall / (k * batch) * 1e6
rows = []
with torch.inference_mode():
    acc = (model(torch.tensor(Xte)).argmax(1).numpy() == yte).mean()
    p99, thr, uj = olchov(lambda x: model(torch.from_numpy(x)), 32)
torch.save(model.state_dict(), "m.pt")
rows.append(("PyTorch eager FP32", "2 oqim", acc, p99, thr, uj, os.path.getsize("m.pt") /
1024))
for nom, f, intra in [("ORT FP32", "fp32.onnx", 1), ("ORT FP32", "fp32.onnx", 2), ("ORT INT8
dinamik", "dyn.onnx", 1),
                      ("ORT INT8 statik", "int8.onnx", 1), ("ORT INT8 statik", "int8.onnx",
2)]:
    so = ort.SessionOptions(); so.intra_op_num_threads = intra
```

```
    s = ort.InferenceSession(f, so, providers=["CPUExecutionProvider"])
    acc = (s.run(None, {"x": Xte})[0].argmax(1) == yte).mean()
    p99, thr, uj = olchov(lambda x: s.run(None, {"x": x}), 32)
    rows.append((nom, f"{intra} oqim", acc, p99, thr, uj, os.path.getsize(f) / 1024))
T = pd.DataFrame(rows, columns=["Konfiguratsiya", "Oqim", "Aniqlik", "p99 b1, ms", "inf/s b32",
"uJ/inf b32", "Hajm, KB"])
print("=== Yakuniy jadval ===")
print(T.round(4).to_string(index=False))

# --- Pareto: kechikish (min), energiya (min), aniqlik (max), throughput (max) — hech kim uni
barcha mezonda ustun bo'lmasa, optimal
def pareto(df):
    v = np.c_[df["p99 b1, ms"], df["uJ/inf b32"], -df["Aniqlik"], -df["inf/s b32"]]
    return [not any((v[j] <= v[i]).all() and (v[j] < v[i]).any() for j in range(len(v)) if j !=
i) for i in range(len(v))]
T["Pareto"] = pareto(T)
print("\n=== Pareto tahlili (p99, energiya, aniqlik, throughput) ===")
print(T[["Konfiguratsiya", "Oqim", "Pareto"]].to_string(index=False))
print("Pareto-optimal:", "; ".join(T[T.Pareto].Konfiguratsiya + " (" + T[T.Pareto].Oqim + ")"))

# --- Ssenariylar: cheklovlar + maqsad funksiyasi
acc0 = T.Aniqlik.iloc[1]
SSEN = [("Bilakuzuk (yiqilish signali)", "hajm <= 20 KB, aniqlik >= FP32-1 p.p.", lambda d:
d[(d["Hajm, KB"] <= 20) & (d.Aniqlik >= acc0 - 0.01)], "uJ/inf b32", True),
        ("Kasalxona serveri (1000 bemor)", "aniqlik >= FP32-0,5 p.p.", lambda d: d[d.Aniqlik >=
acc0 - 0.005], "inf/s b32", False),
        ("Real vaqt signali", "p99 eng kichik", lambda d: d, "p99 b1, ms", True)]
print("\n=== Ssenariy bo'yicha tanlov ===")
for nom, shart, filt, mezon, kichik in SSEN:
    d = filt(T)
    if len(d) == 0: print(f"{nom}: shartni ({shart}) hech qaysi konfiguratsiya
qanoatlantirmaydi"); continue
    tan = d.loc[d[mezon].idxmin() if kichik else d[mezon].idxmax()]
    print(f"{nom}: [{shart}] -> {tan.Konfiguratsiya} ({tan.Oqim}), {mezon} = {tan[mezon]:.3f}")

print("\n=== TensorRT / OpenVINO yo'l xaritasi (GPU/NPU li stendda bajariladi) ===")
for q in ["TensorRT: trtexec --onnx=fp32.onnx --fp16 --saveEngine=har_fp16.plan  (INT8 uchun --
int8 + kalibrlash keshi)",
          "TensorRT: trtexec --loadEngine=har_fp16.plan --shapes=x:32x3x128 --dumpProfile  ->
latency/throughput, qatlam profili",
          "OpenVINO: ovc fp32.onnx  -> IR (.xml/.bin); benchmark_app -m fp32.xml -d CPU -hint
latency | throughput",
          "OpenVINO INT8: NNCF post-training quantization (kalibrlash to'plami ~300 namuna)",
          "Quvvat: nvidia-smi --query-gpu=power.draw --format=csv -lms 100  |  Jetson:
tegrastats  |  CPU: RAPL (perf stat -e power/energy-pkg/)"]:
    print(" -", q)
print("Eslatma: bu labdagi energiya — model asosidagi baho; real tizimda yuqoridagi asboblar
yoki tashqi quvvat o'lchagich bilan tekshiring.")

plt.figure(figsize=(6.5, 4))
for i, r in T.iterrows():
    plt.scatter(r["p99 b1, ms"], r["uJ/inf b32"], s=4000 * (r.Aniqlik - 0.85), c="tab:red" if
r.Pareto else "tab:gray", alpha=0.7)
    plt.annotate(f"{r.Konfiguratsiya} ({r.Oqim})", (r["p99 b1, ms"], r["uJ/inf b32"]),
fontsize=7)
plt.xscale("log"); plt.xlabel("p99 kechikish, ms (batch 1)"); plt.ylabel("uJ/inferens
(taxminiy)")
plt.title("Pareto: qizil — optimal, doira o'lchami — aniqlik"); plt.tight_layout()
```

Natijada quyidagi satr(lar) chiqishi kerak: `=== Yakuniy jadval ===`, `=== Pareto tahlili`, `=== Ssenariy bo'yicha tanlov ===`, `=== TensorRT / OpenVINO yo'l xaritasi`

TAJRIBA QILIB KO‘RING

- Ssenariyga «aniqlik >= FP32» shartini qo'shing — tanlov o'zgaradimi?
- Pareto'ni faqat (p99, energiya) juftligida hisoblang va natijani solishtiring.
- HARNet(w=16) ni qo'shib, kichikroq model Pareto chegarasiga chiqadimi tekshiring.

Mening natijam (o‘lchovlar, kuzatuvlar, xulosa):

Qo‘shimcha mashqlar (10 ta, jami 100 ball)

№	Mashq	Daraja	Ball
1	**O'lchov barqarorligi** 1-qadamda isinish soni (0, 10, 50, 200) va takror soni (50, 400, 2000) kombinatsiyalari uchun p50, p99 va CV ni 5 martadan o'lchang. Jadval va 3–5 jumlali xulosada ishonchli o'lchov uchun minimal sozlamalarni asoslang.	oson	5
2	**Operator profilining batch'ga bog'liqligi** 2-qadam profilini batch 1, 8, 64 va 256 da bajaring. Har operator ulushini ustunli grafikda ko'rsating va batch oshganda qaysi operator(lar) ulushi o'sishini, qat'iy ustama qanday kamayishini izohlang.	oson	5
3	**Format tanlovi jadvali** 3-qadam natijalari asosida 6 format uchun «hajm / aniqlik / moslik / tezlik» jadvalini tuzing va har formatni qaysi apparatda (Tensor Core'li GPU — TensorRT, VNNI li Intel CPU — OpenVINO, ARM chekka qurilma) ishlatish mantiqli ekanini 1–2 jumladan asoslang.	oson	5
4	**Kalibrlash usuli va to'plami** 3-qadamda statik INT8 ni MinMax, Percentile (99,9 / 99,99 / 99,999) va Entropy usullari bilan, 32, 128, 320 va 1000 namunali kalibrlash to'plamlarida bajaring. Aniqlik, FP32 bilan moslik va KL ni jadvalga yozing va eng barqaror kombinatsiyani tanlang.	o'rta	10
5	**Dinamik paketlashni sozlash** 4-qadam simulyatorida max_b (4, 16, 64) va max_wait (0,1; 0,3; 1; 3 ms) panjarasini bu CPU va GPU uslubidagi S(b) uchun bajaring. p99 <= 2 ms SLO ostida maksimal yuklamani beradigan sozlamani toping va natijani issiqlik xaritasi (heatmap) bilan ko'rsating.	o'rta	10
6	**Parallellik va energiya birgalikda** 5-qadam konfiguratsiyalari uchun 6-qadam quvvat modeli bilan µJ/inferens va EDP ni hisoblang. «Eng tez» va «eng tejamkor» konfiguratsiya bir xilmi? Natijani jadval va 1 ta grafik bilan bering, OpenVINO LATENCY/THROUGHPUT ko'rsatmalari bilan bog'lab izohlang.	o'rta	10
7	**Model kengligi bo'yicha masshtablash** HARNet(w) ni w = 8, 16, 32, 64, 128 uchun o'qitib (3-qadam usulida), har biri uchun aniqlik, parametrlar, FP32 va INT8 statik p99 va throughput ni o'lchang. Aniqlik–kechikish egri chizig'ini chizing va «yetarli darajada kichik» modelni tanlang.	o'rta	10
8	**Real quvvat o'lchashga tayyorgarlik** 6-qadamdagi model asosidagi energiya bahosini real o'lchov bilan solishtirish metodikasini ishlab chiqing: o'z kompyuteringizda (agar mavjud bo'lsa) RAPL (perf stat -e power/energy-pkg/) yoki tashqi quvvat o'lchagich bilan FP32 va INT8 uchun J/inferens ni o'lchang, yoki o'lchash imkoni bo'lmasa, quvvat modelining sezgirlik tahlilini (P_bosh, P_max ±50%) bajaring. Xatolik manbalari va noaniqlik intervali bilan hisobot bering.	qiyin	15

9	**Navbat nazariyasi bilan tekshirish** 4-qadamdagi paketsiz siyosat uchun M/D/1 navbat formulasidan (Pollaczek–Khinchine) o'rtacha kutish vaqtini hisoblang va simulyatsiya natijasi bilan 5 ta yuklama nuqtasida solishtiring. Dinamik paketlash uchun nazariy throughput chegarasini 1/c1 orqali chiqaring va simulyatsiyadagi to'yinish bilan solishtiring; tafovutlarni izohlang.	qiyin	15
10	**To'liq optimallashtirish hisoboti** O'zingiz tanlagan boshqa model va ma'lumot (masalan, sklearn digits uchun MLP yoki sintetik EKG uchun 1D CNN) uchun 1–7 qadamlarning barchasini takrorlang: o'lchov metodikasi, eksport va profil, kamida 4 format, paket/oqim tahlili, parallellik, energiya va DVFS, Pareto va ssenariy tanlovi. 4–6 betlik hisobotda «TensorRT yoki OpenVINO bo'lsa nima o'zgaradi» bo'limini buyruqlar bilan yozing.	qiyin	15

Mustaqil topshiriqlar — 20 ta variant

Variant jurnal raqami bo‘yicha tanlanadi (20 dan katta bo‘lsa — 20 ga bo‘lgandagi qoldiq, 0 → 20). Laboratoriya bosqichlari kodini asos qilib, o‘z variantingiz parametrlari bilan bajaring.

1-variant. Tibbiyot: EKG aritmiya detektori — ONNX eksport va o'lchov metodikasi oson

Soha: Tibbiyot (kardiologiya)

Portativ EKG qurilmasi 1 kanalli signalni 360 Hz da yozadi va 1 soniyalik oyna bo'yicha aritmiyani aniqlashi kerak. Qurilma ishlab chiqaruvchisi ONNX formatini qabul qiladi va p99 kechikish 10 ms dan oshmasligini talab qiladi. Sizdan modelni o'qitish, to'g'ri o'lchov metodikasi bilan kechikishni o'lchash va ORT darajalarini solishtirish so'raladi.

Ma'lumot	sintetik EKG (sinus + QRS shakli), 4 sinf
Kirish	1 x 360
Model	1D CNN 3 qatlam (16-32-32)
Batch	1, 8
SLO	p99 <= 10 ms
Takror	5 x 500

QADAMLAR

1. Lab s1 kabi sintetik EKG generatori va 1D CNN ni o'qiting, aniqlikni bering.
2. Isinish va takror bilan PyTorch eager va TorchScript kechikishini (p50/p99/CV) o'lchang.
3. Lab s2 kabi ONNX eksport va 4 ORT darajasi: tugunlar, operatorlar, p50/p99.
4. ORT profil bilan eng og'ir operatorni toping.
5. Xulosa: SLO bajariladimi, qaysi daraja va rejim tavsiya etiladi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

2-variant. Agro: tuproq namligi sensorlari bo'yicha sug'orish tasnifi — INT8 dinamik oson

Soha: Agro

Dala sensori (8 ta o'lchov: namlik, harorat, EC, ...) bo'yicha sug'orish kerak/kerak emas qarori Raspberry Pi'da qabul qilinadi. Model kichik bo'lishi va kam energiya sarflashi kerak. Sizdan MLP ni INT8 dinamik kvantlash, hajm, aniqlik va tezlikni solishtirish so'raladi.

Ma'lumot	make_classification (8 belgi, 3 sinf, 5000)
Model	MLP 8-128-64-3
Kvantlash	PyTorch va ORT dinamik INT8
Batch	1, 64
Maqsad hajm	< 20 KB
Dvigatel	qnnpack

QADAMLAR

1. MLP ni o'qitib, FP32 aniqlik, hajm va p50 ni o'lchang.
2. PyTorch quantize_dynamic (qnnpack) bilan INT8: hajm, aniqlik, tezlik.
3. ONNX eksport va ORT quantize_dynamic: xuddi shu o'lchovlar.
4. Lab s3 kabi FP32 bilan moslik va KL ni hisoblang.
5. Xulosa: Raspberry Pi uchun qaysi variant (ARM'da INT8 qanday bo'lishi mumkin — taxminiy izoh).

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

3-variant. Transport: yo'l belgilari tasnifi CNN — statik INT8 va kalibrlash o'rta

Soha: Transport

Avtomobil kamerasi uchun 8x8 dan kattaroq (16x16) sintetik yo'l belgilari tasvirlarini tasniflovchi CNN optimallashtirilmoqda. Aniqlik 1 p.p. dan ko'p tushmasligi kerak. Sizdan statik INT8 ni uch kalibrlash usuli va ikki format (QDQ, QOperator) bilan sinash so'raladi.

Ma'lumot	sintetik 16x16 shakllar (doira, uchburchak, kvadrat, ...), 6 sinf
Model	CNN 2 Conv + 1 FC
Kalibrlash	MinMax, Percentile 99,99, Entropy
Format	QDQ, QOperator
Kalibrlash to'plami	64 va 512
Chegara	aniqlik pasayishi <= 1 p.p.

QADAMLAR

1. Sintetik tasvir generatori va CNN ni o'qiting, ONNX'ga eksport qiling.
2. Lab s3 kabi 3 kalibrlash usuli x 2 format x 2 to'plam hajmi bilan statik INT8 yarating.
3. Har biri uchun aniqlik, moslik, KL, hajm va throughput (batch 64) ni jadvalga yozing.
4. Operatorlar ro'yxatini (QLinearConv, Q/DQ) taqqoslang.
5. Talabni qanoatlantiradigan eng tez konfiguratsiyani tanlang.
6. Xulosa: TensorRT INT8 kalibrlash keshi bilan bog'lab izohlang.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

4-variant. Telekom: trafik anomaliyasi autoenkoderi — paket va throughput o'rta

Soha: Telekom

Operator tarmoq oqimlarini (20 belgi) real vaqtda tekshiradi; soniyasiga 50 000 oqim keladi. Autoenkoder rekonstruksiya xatosi bo'yicha anomaliyani aniqlaydi. Sizdan batch bo'yicha S(b) modelini qurish, dinamik paketlash simulyatsiyasi bilan kerakli nusxalar sonini topish so'raladi.

Ma'lumot	sintetik oqim belgilari (20), 2% anomaliya
Model	Autoenkoder 20-64-8-64-20
Batch	1–1024
Yuklama	50 000 oqim/s
SLO	p99 <= 5 ms
Siyosat	paketsiz, b<=128 kutish 1 ms

QADAMLAR

1. Autoenkoderni o'qiting va anomaliya aniqlash ROC-AUC sini bering.
2. Lab s4 kabi batch 1–1024 da S(b) ni o'lchab, c0 va c1 ni toping.
3. Dinamik paketlash simulyatsiyasida yuklama 10k–60k uchun p99 va throughput.
4. SLO ni saqlash uchun kerakli nusxalar sonini hisoblang.
5. Lab s5 kabi 2 sessiya x 1 oqim konfiguratsiyasi bilan tekshiring.
6. Xulosa: paketlash va nusxalar soni bo'yicha tavsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

5-variant. Energetika: quyosh paneli quvvati prognozi — regressiyada kvantlash xatosi oson

Soha: Energetika

Quyosh elektr stansiyasi har 15 daqiqada keyingi soat quvvatini prognoz qiladi; model kichik kontrollerda ishlaydi. Regressiyada kvantlash xatosi to'g'ridan-to'g'ri MAE ga ta'sir qiladi. Sizdan FP32 va INT8 variantlarda MAE va kechikishni solishtirish so'raladi.

Ma'lumot	sintetik quyosh nurlanishi + bulut (24 belgi)
Model	MLP 24-64-32-1
Kvantlash	ORT dinamik va statik
Metrika	MAE (kW), max xato
Batch	1
Kalibrlash	200 namuna

QADAMLAR

1. Sintetik ma'lumot va MLP regressorni o'qiting, MAE ni bering.
2. ONNX eksport va ORT FP32 kechikishi.
3. Dinamik va statik INT8: MAE, max xato, hajm, kechikish.
4. Kvantlash xatosining taqsimotini gistogrammada ko'rsating.
5. Xulosa: regressiyada INT8 ishlatish mumkinmi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

6-variant. Moliya: kredit skoring — daraxtli modeldan MLP distillyatsiyasi va energiya qiyin

Soha: Moliya

Bank skoring modeli (gradient boosting) ONNX'ga to'g'ridan-to'g'ri eksport qilinmaydi (skl2onnx yo'q). Yechim — bilimni MLP ga distillyatsiya qilish va uni optimallashtirish. Sizdan distillyatsiya sifati, ORT optimallashtirish, INT8, paket tahlili va energiya bahosi bilan to'liq hisobot so'raladi.

Ma'lumot	make_classification (30 belgi, 20 000, 10% defolt)
O'qituvchi	HistGradientBoosting
Talaba	MLP 30-128-64-2
Metrika	ROC-AUC, o'qituvchi bilan moslik
Batch	1, 256
Energiya	P_bosh 10 W, P_max 45 W (faraz)

QADAMLAR

1. Boosting modelini o'qiting va ROC-AUC ni bering.
2. O'qituvchi ehtimollari (soft label) bo'yicha MLP ni distillyatsiya qiling; AUC va moslikni solishtiring.
3. MLP ni ONNX'ga eksport qilib, ORT darajalari va profilni bajaring.
4. INT8 dinamik/statik: AUC, hajm, throughput.
5. Lab s4 kabi S(b) va dinamik paketlash (yuklama 5000 so'rov/s).
6. Lab s6 kabi µJ/inferens va EDP ni barcha variantlar uchun hisoblang.
7. Xulosa: Pareto va server uchun tanlov.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar, statistik asoslash — 30, Tahlil, murosa va tavsiyalar — 30, Hisobot sifati — 15

7-variant. Ta'lim: kichik transformer bloki — MatMul kvantlash va parallellik

qiyin

Soha: Ta'lim

Ta'lim platformasi talabaning 32 ta oxirgi javobi ketma-ketligi bo'yicha keyingi javob to'g'riligini bashorat qiladi (knowledge tracing). Model — bitta transformer bloki. Sizdan eksport, fusion, MatMul kvantlash, parallellik konfiguratsiyalari va energiyani tahlil qilish so'raladi.

Ma'lumot	sintetik javoblar ketma-ketligi (32 qadam, 50 ko'nikma)
Model	Embedding 64 + TransformerEncoderLayer (4 bosh) + Linear
Kvantlash	dinamik INT8 (MatMul)
Parallellik	intra 1/2, 2 sessiya
Batch	1, 32
SLO	p99 <= 3 ms

QADAMLAR

1. Sintetik ketma-ketliklar va modelni o'qiting, AUC ni bering.
2. ONNX (opset 17) eksport; ORT ALL darajasidagi operatorlarni va fusion'ni (Attention/LayerNorm) tahlil qiling.
3. Dinamik INT8: AUC, hajm, tezlik (MatMul og'ir modelda farq CNN dan kattami).
4. Lab s5 kabi 5 parallellik konfiguratsiyasi batch 1 va 32 da.
5. Lab s6 kabi energiya bahosi.
6. Xulosa: TensorRT da transformer uchun FP16 va plaginlar haqida izoh.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar, statistik asoslash — 30, Tahlil, murosa va tavsiyalar — 30, Hisobot sifati — 15

8-variant. Sanoat: podshipnik vibratsiyasi 1D CNN — profil va sezgir qatlam

o'rta

Soha: Sanoat

Zavod nasoslaridagi vibratsiya sensori (2 kHz, 512 nuqta) bo'yicha nosozlik turini aniqlash modeli sanoat kompyuterida ishlaydi. Sizdan operator profili orqali og'ir qatlamni topish va statik INT8 da sezgir qatlamni FP32 da qoldirish tahlilini bajarish so'raladi.

Ma'lumot	sintetik vibratsiya (asosiy chastota + nosozlik garmonikalari), 4 sinf
Kirish	1 x 512
Model	1D CNN 4 qatlam
Kvantlash	statik QDQ, nodes_to_exclude
Profil	batch 1 va 64
Chegara	aniqlik pasayishi <= 0,5 p.p.

QADAMLAR

1. Generator va 1D CNN ni o'qiting.
2. Lab s2 kabi profil bilan operatorlar vaqt ulushini batch 1 va 64 da toping.
3. To'liq statik INT8 aniqligi va moslikni o'lchang.
4. Har Conv ni navbatma-navbat FP32 da qoldirib, aniqlikni o'lchang.
5. «INT8 + 1 FP32 qatlam» va to'liq INT8 ni tezlik/aniqlik bo'yicha solishtiring.
6. Xulosa: sezgir qatlam va uning sababi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

9-variant. Kiberxavfsizlik: login hodisalari MLP — keng model va hajm oson

Soha: Xavfsizlik

Kirish tizimi har login urinishini (16 belgi) tekshiradi. Model xotirada kichik bo'lishi kerak, chunki u har serverda ishlaydi. Sizdan kenglik bo'yicha (128, 512) ikki MLP ni INT8 dinamik kvantlash va hajm nisbatini tahlil qilish so'raladi.

Ma'lumot	make_classification (16 belgi, 3% hujum)
Modellar	MLP 16-128-2 va 16-512-512-2
Kvantlash	ORT dinamik INT8
Metrika	recall, hajm nisbati
Batch	1, 128

QADAMLAR

1. Ikki MLP ni o'qiting va recall ni bering.
2. ONNX eksport va FP32 hajm/kechikish.
3. Dinamik INT8: hajm nisbati (4x ga qanchalik yaqin), recall, tezlik.
4. Natijalarni jadvalda bering.
5. Xulosa: keng modelda kvantlash foydasi nega kattaroq.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

10-variant. Chakana savdo: embedding + MLP tavsiya modeli — katta paket o'rta

Soha: Chakana savdo

Onlayn do'kon tavsiya nomzodlarini qayta saralovchi modelni bir so'rovda 200 ta nomzod uchun ishlatadi. Shuning uchun katta paket (200–1000) asosiy rejim. Sizdan embedding (Gather) va MLP li modelni eksport qilish, katta paketda throughput va kvantlash ta'sirini o'lchash so'raladi.

Ma'lumot	sintetik foydalanuvchi-mahsulot juftlari (5000 mahsulot)
Model	Embedding 5000x32 + MLP 64-64-1
Batch	200, 500, 1000
Kvantlash	dinamik INT8 (Gather va MatMul)
SLO	200 nomzod <= 5 ms
Metrika	AUC, nDCG@10

QADAMLAR

1. Modelni o'qiting, AUC va nDCG@10 ni bering.
2. ONNX eksport; embedding jadvali hajmi umumiy hajmning qancha qismi.
3. Batch 200/500/1000 da S(b) va throughput.
4. Dinamik INT8: hajm, AUC, nDCG, tezlik.
5. Lab s5 kabi intra=1/2 bilan SLO bajarilishi.
6. Xulosa: katta paket rejimida qaysi konfiguratsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

11-variant. Logistika: qo'l skanerida yetkazish vaqti prognozi — energiya va batareya o'rta

Soha: Logistika

Kuryerning qo'l skaneri (ARM, batareya 3000 mAh) har buyurtma uchun yetkazish vaqtini prognoz qiladi. Batareya bir smenaga (10 soat) yetishi kerak. Sizdan energiya modelini skaner parametrlariga moslash, FP32/INT8 da J/inferens va batareya umrini hisoblash so'raladi.

Model	MLP 12-64-32-1
Inferens chastotasi	har 2 soniyada 1 (fon) + 200 buyurtma
Quvvat modeli	P_bosh 0,4 W, P_max 2,5 W (faraz)
Batareya	3000 mAh x 3,7 V
Format	FP32, INT8 statik
DVFS	0,6–1,8 GHz

QADAMLAR

1. Modelni o'qiting, ONNX va INT8 statik variantlarni yarating.
2. Lab s6 kabi bandlik va µJ/inferens ni ikkala format uchun o'lchang.
3. Skaner quvvat modeliga moslab smena davomida model iste'mol qiladigan energiyani hisoblang.
4. DVFS simulyatsiyasida eng tejamkor chastotani toping (SLO 20 ms).
5. Batareya ulushini (model / butun qurilma) baholang.
6. Xulosa: model optimallashtirishning batareya umriga real ta'siri (taxminiy).

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

12-variant. Shahar: chorraha tiqilinchi bashorati — oqimlar va Little qonuni oson

Soha: Shahar

Shahar trafik markazi 300 chorraha uchun har 5 soniyada tiqilinch darajasini bashorat qiladi (soniyasiga 60 so'rov). Sizdan kichik modelni ORT da o'lchash va Little qonuni bilan nechta nusxa kerakligini hisoblash so'raladi.

Ma'lumot	sintetik trafik matritsasi (12 belgi)
Model	MLP 12-32-3
Yuklama	60 so'rov/s (cho'qqida 600)
Oqimlar	intra 1, 2
SLO	p99 <= 20 ms

QADAMLAR

1. Modelni o'qiting va ONNX'ga eksport qiling.
2. ORT intra=1 va 2 da p50/p99 ni o'lchang.
3. Little qonuni (N = X·R) bilan oddiy va cho'qqi yuklama uchun nusxalar sonini hisoblang.
4. Natijani jadvalda bering.
5. Xulosa: bu yuklama uchun optimallashtirish kerakmi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

13-variant. Ekologiya: havo sifati tasnifi — INT8 va siyraklashtirish (pruning)

o'rta

Soha: Ekologiya

Shahar havo sifati stansiyasi (10 sensor) har daqiqada havo sifati sinfini aniqlaydi. Model mikrokompyuterda ishlaydi. Sizdan INT8 kvantlash va magnituda bo'yicha siyraklashtirishni (50%, 80%) hajm, aniqlik va tezlik bo'yicha taqqoslash so'raladi.

Ma'lumot	sintetik sensor o'lchovlari, 5 sinf
Model	MLP 10-256-128-5
Pruning	torch.nn.utils.prune 50% va 80%
Kvantlash	ORT statik INT8
Metrika	aniqlik, nol vaznlar ulushi, hajm (zip bilan)
Batch	1, 64

QADAMLAR

1. Modelni o'qiting.
2. 50% va 80% magnituda pruning; aniqlik va fine-tune (1 epoxa) dan keyingi aniqlik.
3. ONNX eksport; siqilgan (zip) hajm va kechikish — zich yadroda pruning tezlik beradimi?
4. Statik INT8 va «pruning + INT8» kombinatsiyasi.
5. Barcha variantlar jadvali va Pareto grafigi.
6. Xulosa: TensorRT 2:4 siyraklik va OpenVINO siyrak yadrolari haqida izoh.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

14-variant. Sport: akselerometr + giroskop harakat tanish — CNN+GRU, energiya va Pareto qiyin

Soha: Sport

Sport soati 6 kanalli (akselerometr + giroskop) signal bo'yicha mashq turini aniqlaydi. Model qurilmada uzluksiz ishlaydi, shuning uchun energiya asosiy mezon. Sizdan CNN+GRU modelini eksport qilish, statik kvantlash (per-tensor vs per-channel), parallellik, DVFS va Pareto tahlili so'raladi.

Ma'lumot	sintetik 6 x 100 signal, 8 mashq turi
Model	Conv1d 32 + GRU 32 + Linear
Kvantlash	statik QDQ per-tensor va per-channel, dinamik
Batch	1
Energiya	P_bosh 0,05 W, P_max 0,6 W (faraz)
SLO	p99 <= 15 ms

QADAMLAR

1. Lab s1 kabi generator va CNN+GRU ni o'qiting; o'lchov metodikasi bilan eager kechikishini o'lchang.
2. ONNX eksport (GRU operatori) va profil — GRU ulushi.
3. Per-tensor, per-channel statik va dinamik INT8: aniqlik, moslik, KL, tezlik.
4. Lab s5 kabi parallellik (intra 1/2) va lab s6 kabi µJ/inferens.
5. DVFS simulyatsiyasida soat protsessori uchun eng tejamkor chastota.
6. Lab s7 kabi Pareto va soat uchun tanlov.
7. Xulosa: qurilmada (NPU bo'lsa) nima o'zgarishi mumkin.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar, statistik asoslash — 30, Tahlil, murosa va tavsiyalar — 30, Hisobot sifati — 15

15-variant. Media: sharh kayfiyati so'zlar sumkasi MLP — dinamik kvantlash

oson

Soha: Media

Kino portali foydalanuvchi sharhlarining kayfiyatini (ijobiy/salbiy/neytral) aniqlaydi. Matn 2000 so'zli lug'at bo'yicha so'zlar sumkasi vektoriga aylantiriladi. Sizdan MLP ni dinamik kvantlash va hajm/tezlikni solishtirish so'raladi.

Ma'lumot	sintetik sharhlar (lug'at 2000 so'z)
Model	MLP 2000-128-3
Kvantlash	ORT dinamik INT8
Batch	1, 32
Metrika	aniqlik, hajm, p50

QADAMLAR

1. Sintetik sharh generatori va MLP ni o'qiting.
2. ONNX eksport, FP32 hajm va kechikish.
3. Dinamik INT8: hajm (~4x), aniqlik, tezlik.
4. Natijalar jadvali.
5. Xulosa: birinchi qatlam (2000x128) hajmning qancha qismi va nega.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

16-variant. Tibbiyot: implant yurak monitori — uch mezonli Pareto va hajm chegarasi o'rta

Soha: Tibbiyot (implant)

Teri ostiga o'rnatiladigan yurak monitori 64 KB xotiraga ega va batareyasi 3 yil ishlashi kerak. Sizdan EKG oynasi bo'yicha 1D CNN ning bir nechta kenglikdagi variantlarini FP32/INT8 da o'lchab, hajm cheklovi ostida aniqlik–energiya Pareto chegarasini topish so'raladi.

Ma'lumot	sintetik EKG oynalari (1 x 256), 3 sinf
Modellar	1D CNN w = 4, 8, 16, 32
Format	FP32, INT8 statik
Hajm chegarasi	<= 64 KB
Energiya	quvvat modeli (faraz)
Metrika	sezgirlik (recall) >= 0,95

QADAMLAR

1. Generator va 4 kenglikdagi modellarni o'qiting.
2. Har biri uchun FP32 va INT8 statik ONNX yarating.
3. Hajm, recall, kechikish va µJ/inferens ni o'lchang.
4. Hajm <= 64 KB va recall >= 0,95 shartlari ostida Pareto chegarasini toping.
5. Tanlangan model uchun 3 yillik energiya byudjetini taxminiy hisoblang.
6. Xulosa.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

17-variant. Aviatsiya: dvigatel qoldiq resursi (RUL) prognozi — ONNX va energiya

o'rta

Soha: Aviatsiya

Samolyot bort kompyuteri dvigatelning 14 sensori bo'yicha qoldiq resursni (sikllar) prognoz qiladi. Sertifikatlash talablari tufayli FP32 dan farq kichik bo'lishi kerak. Sizdan regressiya modelini optimallashtirish va energiya/kechikishni baholash so'raladi.

Ma'lumot	sintetik degradatsiya egri chiziqlari (14 sensor, 30 qadam oyna)
Model	1D CNN + Linear (regressiya)
Format	FP32, FP16 vazn (simul.), INT8 statik
Metrika	RMSE (sikl), max |farq| FP32 dan
Chegara	max |farq| <= 2 sikl
Batch	1

QADAMLAR

1. Generator va regressorni o'qiting, RMSE ni bering.
2. ONNX eksport, ORT darajalari va profil.
3. FP16 vazn simulyatsiyasi va INT8 statik: RMSE va FP32 dan max farq.
4. Lab s6 kabi energiya bahosi va EDP.
5. Chegarani qanoatlantiradigan eng tejamkor format.
6. Xulosa: sertifikatlashda kvantlash xavfi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar va jadval — 25, Tahlil va xulosa — 30, Hisobot sifati — 20

18-variant. Fintech: to'lov firibgarligi — kvantlashning recall va chegaraga ta'siri

qiyin

Soha: Fintech

To'lov tizimi har tranzaksiyani 2 ms ichida tekshirishi kerak; soniyasiga 20 000 tranzaksiya. Kvantlash ehtimollarni siljitib, chegaradagi recall ni o'zgartirishi mumkin. Sizdan kvantlashning qaror chegarasiga ta'sirini, dinamik paketlash va nusxalar rejasini to'liq tahlil qilish so'raladi.

Ma'lumot	make_classification (40 belgi, 0,3% firibgarlik)
Model	MLP 40-256-128-1
Kvantlash	statik MinMax, Percentile, Entropy
Metrika	recall@FPR 0,5%, PR-AUC
Yuklama	20 000 so'rov/s
SLO	p99 <= 2 ms

QADAMLAR

1. Modelni o'qiting, FP32 PR-AUC va recall@FPR 0,5% ni bering.
2. Uch kalibrlash usuli bilan statik INT8: PR-AUC, recall, chegara siljishi.
3. Kvantlangan model uchun chegarani qayta kalibrlab, recall tiklanishini ko'rsating.
4. Lab s4 kabi S(b) va dinamik paketlash simulyatsiyasi 20 000 so'rov/s da.
5. Lab s5 kabi parallellik va kerakli nusxalar soni.
6. Lab s6 kabi energiya va yakuniy Pareto.
7. Xulosa: kvantlashdan keyin chegarani qayta tekshirish qoidasi.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar, statistik asoslash — 30, Tahlil, murosa va tavsiyalar — 30, Hisobot sifati — 15

19-variant. Oziq-ovqat sanoati: konveyerda nuqsonli mahsulot — ORT va 500 dona/s oson

Soha: Oziq-ovqat sanoati

Konveyer kamerasi har mahsulotni (sintetik 24x24 kulrang tasvir) yaroqli/nuqsonli deb ajratadi; tezlik soniyasiga 500 dona. Sizdan kichik CNN ni ORT da o'lchash va talab bajarilishini tekshirish so'raladi.

Ma'lumot	sintetik 24x24 tasvirlar (dog', yoriq), 2 sinf
Model	CNN 2 Conv (8-16) + FC
Batch	1, 16
Talab	500 dona/s, p99 <= 10 ms
Oqimlar	intra 1, 2

QADAMLAR

1. Generator va CNN ni o'qiting, aniqlik va recall (nuqsonli) ni bering.
2. ONNX eksport va ORT ALL darajasida p50/p99.
3. Batch 1 va 16 da throughput.
4. intra=1 va 2 da talab bajarilishini tekshiring.
5. Xulosa: bitta CPU yetadimi, zaxira qancha.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 30, O'lchovlar va jadval — 25, Tahlil va xulosa — 25, Hisobot sifati — 20

20-variant. Konchilik: NIR spektr bo'yicha ruda tarkibi — to'liq optimallashtirish zanjiri qiyin

Soha: Konchilik

Boyitish fabrikasida konveyerdagi rudaning NIR spektri (256 to'lqin uzunligi) bo'yicha metall ulushi bashorat qilinadi. Natijalar texnologik jarayonni boshqaradi, shuning uchun aniqlik, kechikish va ishonchlilik muhim. Sizdan to'liq optimallashtirish zanjiri va natijalarning ishonch intervallarini berish so'raladi.

Ma'lumot	sintetik spektrlar (3 mineral aralashmasi + shovqin), 256 nuqta
Model	1D CNN regressor
Format	FP32, INT8 dinamik, INT8 statik
Metrika	RMSE (%), R2
O'lchov	5 takror, bootstrap 95% CI
SLO	p99 <= 5 ms, 200 spektr/s

QADAMLAR

1. Lab s1 kabi generator, model va o'lchov metodikasi (5 takror, CV).
2. ONNX eksport, darajalar va operatorlar profili.
3. Uch format: RMSE, R2, moslik, hajm.
4. Lab s4–s5 kabi paket, dinamik paketlash va parallellik tahlili.
5. Lab s6 kabi energiya va DVFS.
6. p50/p99 va RMSE farqlari uchun bootstrap 95% ishonch intervallari; farqlar statistik ahamiyatlimi.
7. Lab s7 kabi Pareto va fabrika uchun yakuniy tavsiya.

Topshiriladi: Kod (.py, laboratoriya qadamlari asosida moslashtirilgan); Natijalar jadvali (CSV yoki hisobotdagi jadval); Grafik(lar) (PNG); Xulosa (kamida 150 so'z, raqamlarga tayangan)

Baholash (100 ball): To'g'ri ishlaydigan kod — 25, O'lchovlar, statistik asoslash — 30, Tahlil, murosa va tavsiyalar — 30, Hisobot sifati — 15